

AI SDR Dashboard

A local console over the Sybrant QuickMail account: it builds campaigns, mirrors every inbound reply, drafts answers, and gates anything that leaves the building behind a human click.

This document covers what the system does, which QuickMail APIs exist and which we actually use, how the reply sync works, and the wall we hit that forced browser automation.

50,546	375	401,428	17
leads mirrored	reply threads	emails sent to date	sending mailboxes

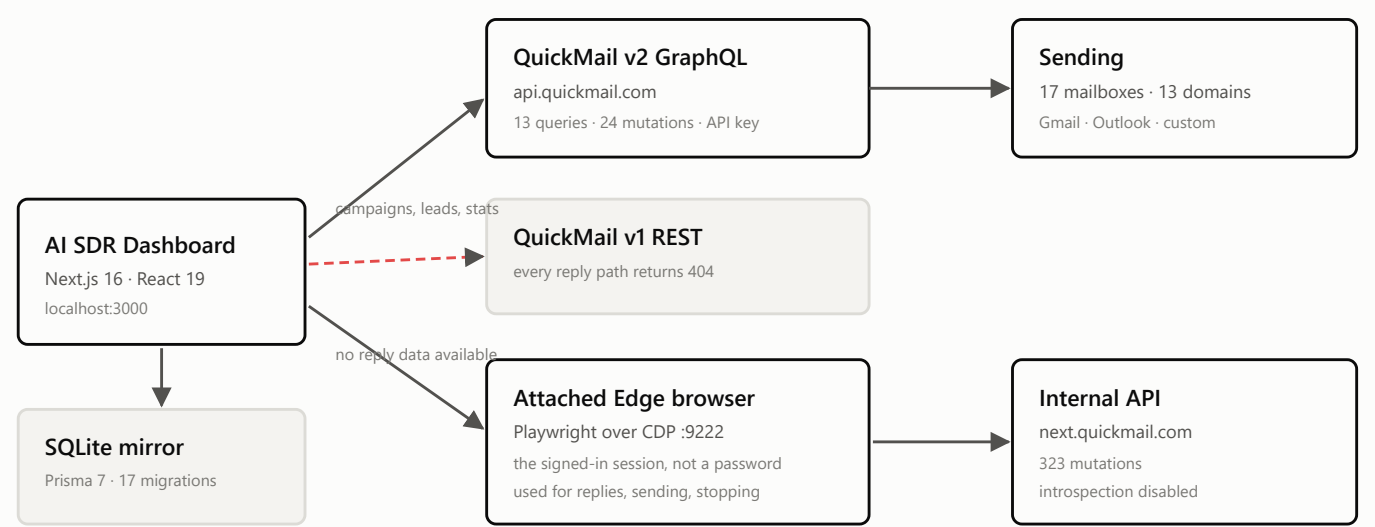
Figures read from the live account and the local mirror on 7 August 2026.

1 • What the system is

QuickMail is the system of record for sending. This app is the orchestration layer around it: it holds the leads, the templates, the AI, and — most importantly — the approvals. Nothing outbound and nothing irreversible happens without a person clicking.

Architecture

Three ways into QuickMail, because one is not enough



Solid lines are routes in use. The dashed red line is the v1 REST API, probed and abandoned — every reply-shaped path returns 404.

Tech stack

LAYER	CHOICE	WHY THIS ONE
Framework	Next.js 16 (App Router)	Server components read the database directly; no separate API tier to keep in step.
Database	SQLite via Prisma 7	Local-first. The whole account fits in one file that can be copied and inspected.
Auth	Auth.js v5, JWT	Internal tool, one workspace; no session table needed.
UI	Tailwind v4 · shadcn/ui	Accessible primitives without a component library to fight.
Browser automation	Playwright over CDP	Attaches to the user's signed-in Edge. No QuickMail password is ever stored.
AI	Pluggable: Gemini · Claude · Ollama · off	Per-task provider, so a dead key degrades one feature instead of all of them.

2 · The QuickMail API surface

Three separate APIs carry the QuickMail name, and they are not equivalent. The published one is small; their own web app uses a far larger private one. That gap is the single most important fact about this integration.

What we are allowed to call vs what their app calls

Mutations available on each surface



The public API cannot read a reply, cannot send an email, and cannot set a campaign trigger. Their own front end does all three.

2.1 · Public v2 GraphQL — `api.quickmail.com/v2/graphql`

Auth: the raw API key in an `Authorization` header — no `Bearer` prefix. Rate limit: 10 requests per 10 seconds, enforced exactly.

QUERY	USED	WHAT WE TAKE FROM IT
<code>campaigns</code> / <code>campaign</code>	yes	359 campaigns: name, paused, steps, and all delivery stats.
<code>leads</code> / <code>lead</code>	yes	50,546 prospects: name, email, company, title, phone, location, tags.
<code>emailAccounts</code>	yes	17 mailboxes with the <code>authorized</code> flag — a dead sender is invisible otherwise.
<code>workspace</code> / <code>workspaces</code>	yes	Workspace 54552, used to scope every other call.
<code>tags</code> , <code>customProperties</code> , <code>agency</code>	no	Available; nothing in the product needs them yet.
MUTATION	USED	PURPOSE
<code>createCampaign</code>	yes	Creates the campaign shell.
<code>createEmailStep</code> · <code>createWaitStep</code>	yes	Builds the sequence. Must end on a wait step — see §5.
<code>setCampaignEmailAccounts</code>	yes	Attaches senders. Returns HTTP 500 for 5 of our 17 mailboxes.
<code>updateCampaignAutomation</code>	yes	Sending window. Needs an IANA zone, not a display label.
<code>createLeads</code> · <code>addLeadsToCampaign</code>	yes	Enrols prospects, after the suppression gate.
<code>updateEmailStep</code>	yes	Pauses individual variations — our stand-in for a campaign schedule.
LinkedIn steps (7 mutations)	no	Not part of this workflow.

MUTATION	USED	PURPOSE
Anything that sends or reads mail	absent	Does not exist on this API.

2.2 · v1 REST — `api.quickmail.io/v1`

Probed endpoint by endpoint for reply data: `/replies`, `/inbox`, `/messages`, `/threads`, `/conversations`, `/emails`, `/activity`, `/unibox`. **All returned 404.** The surface is read-only campaign listing and nothing we need.

2.3 · Internal API — `next.quickmail.com/graphql`

This is what QuickMail's own web app calls. Introspection is disabled, so the operations were read out of their JavaScript bundle rather than discovered by querying. Five are in use:

OPERATION	WHAT IT DOES FOR US
<code>appComponentsWorkspaceOpportunities</code>	Lists reply threads — 374 of them, 30 per page.
<code>workspaceAppOpportunity</code>	One thread in full: every message, both directions, with bodies.
<code>replyToEmail</code>	Sends a reply, threaded, from the mailbox that received it.
<code>setProspectsAsDoNotContact</code>	Bars a prospect from future enrolment.
<code>cancelProspects</code>	Cancels a sequence already in flight.

2.4 · What we send — the write side

Everything above is reading. This is what leaves the app. GraphQL has no GET and POST — every call is an HTTP POST — so the distinction that matters is *query* versus *mutation*, and mutations are the ones that change the account. All of them route through a single gate that refuses to run unless `QUICKMAIL_DRY_RUN=false` is set explicitly.

MUTATION → SENT	PAYLOAD WE BUILD
<code>createCampaign</code> the shell	<code>{ workspaceId, name }</code>
<code>setCampaignEmailAccounts</code> one call per mailbox	<code>{ campaignId, emailAccountIds: [id], assign: true }</code> Sent individually, not as a batch — 5 of our 17 mailboxes return HTTP 500, and a batch would lose the whole campaign to one bad sender.
<code>updateCampaignAutomation</code> the sending window	<code>{ campaignId, timeZone, businessDays: {mon...sun}, timeRanges: [{day, from, to}] }</code> <code>timeZone</code> must be IANA — a display label is rejected, non-fatally.
<code>createEmailStep</code> each email in the sequence	<code>{ campaignId, subject, body, preview, cced, bcced, draft, paused, openTracking }</code> <code>body</code> is HTML. Follow-ups omit <code>subject</code> and send <code>continueThread: true</code> so the reply stays on one thread.
<code>createWaitStep</code> between emails, and at the end	<code>{ campaignId, unit: "days", value, business: true, paused }</code> <code>"days"</code> plural — the singular is rejected.
<code>createLeads</code> prospects QuickMail doesn't have	<code>{ workspaceId, leads: [{ email, firstName, lastName, companyName, title, role, phone, location, score }] }</code>

MUTATION → SENT	PAYLOAD WE BUILD
<code>addLeadsToCampaign</code> enrolment	<code>{ campaignId, leadIds: [...] }</code> Every address passes the suppression gate first. Barred ones never reach this call.
<code>updateEmailStep</code> our stand-in for a schedule	<code>{ stepId, paused }</code> QuickMail has no campaign start or end date, so the scheduler pauses and unpauses each variation.

Writes on the internal API — `replyToEmail` , `setProspectsAsDoNotContact` , `cancelProspects` — are covered in §7, because each of those is gated by a human click rather than by code.

Reads are free; writes are not. Queries run whenever the dashboard needs them, paced to the documented 10-per-10-seconds. Mutations are the opposite: dry-run by default, one campaign at a time, and every campaign id must be listed in `QUICKMAIL_ALLOWED_CAMPAIGN_IDS` before anything can touch it. The key points at a live production workspace with real sending domains and 50,000 real prospects.

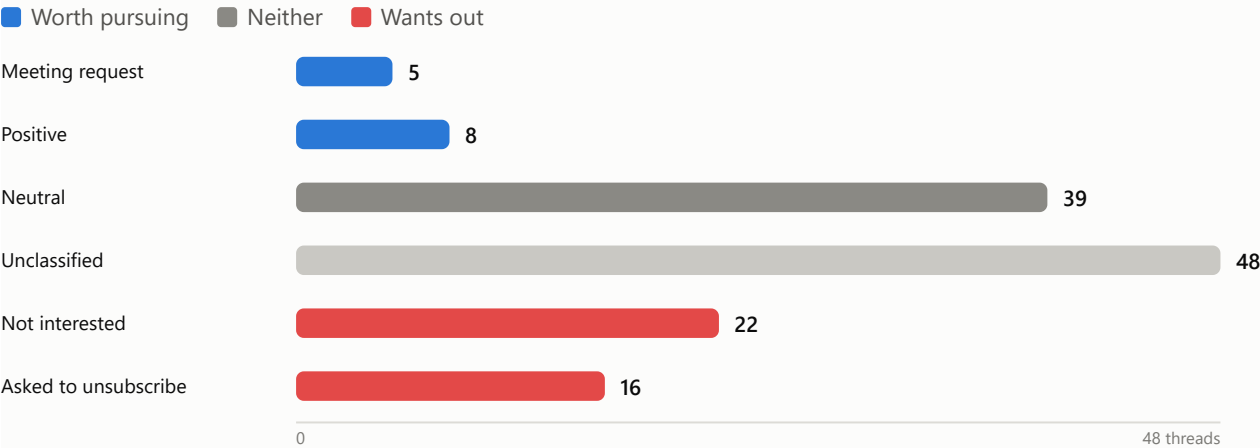
Standing risk. These five are undocumented and unversioned. QuickMail can change them without notice, and when they do, replies and sending break. Everything built on them degrades to a clear error rather than silent failure, and the public API remains the only path for campaign creation.

3 · What we get out of QuickMail

375 reply threads mirrored	1,127 messages, both directions	574 inbound replies	237 of those are auto-replies
DATA	SOURCE	DETAIL	
Campaigns & stats	v2 GraphQL	Sent, delivered, opens, clicks, replies, bounces, unsubscribes — per campaign.	
Leads	v2 GraphQL	50,546 prospects with contact details and tags.	
Mailboxes	v2 GraphQL	17 accounts, each with authorised / paused state.	
Reply threads	Internal API	Subject, HTML body, from / to / cc, timestamps, campaign, receiving mailbox.	
Prospect flags	Internal API	Do-not-contact, and the prospect id needed to set it.	
Reply classification	Ours	QuickMail returns <code>null</code> on nearly every thread; we classify locally.	

What the 138 human replies actually say

Auto-replies excluded · classified locally



38 people have asked to stop — 16 of them explicitly. Each waits on a human click; nothing is suppressed automatically. The 48 unclassified are threads the model never reached, see §6.

4 · How the reply sync works

Replies are **mirrored, not fetched on demand**. The source is an undocumented endpoint behind a browser session: far too slow and too fragile to sit in a page render. The tab reads the local mirror; the sync refreshes it.

One sync run

Roughly 750 round trips · about 10 minutes for the full inbox

- 1

Attach to the signed-in browser
Playwright connects over CDP on :9222 and finds the QuickMail tab. No password is stored.
- 2

Page the thread list
30 per page — the API ignores a larger page size, so paging is explicit. 374 threads total.
- 3

Fetch each thread in full
One request per thread: every message, both directions, with HTML bodies and headers.
- 4

Upsert locally, one thread at a time
Committed as it goes, so a failure at thread 240 keeps the first 239. Never overwrites a human decision.
- 5

Re-read answered threads
Replying moves a thread out of QuickMail's active scope — without this the mirror freezes on the ones you acted on.

Read-only against QuickMail throughout. Connection failures retry four times; a GraphQL error surfaces immediately.

Two ways to run it

ROUTE	SCOPE	WHEN
Sync replies button	newest 30 · ~40s	Day to day, from the Replies tab.
<code>npm run sync-replies -- --limit=400</code>	everything · ~10min	Backfill, or after a long gap.

Both call the same code. Two implementations would eventually disagree about what "synced" means.

What the sync will not do

- **Never suppresses anyone.** QuickMail's do-not-contact flag is mirrored and blocks the Send button, but barring an address is a human decision.
- **Never overwrites a handled thread.** A re-sync cannot resurrect something already dealt with.
- **Never overwrites our classification** with QuickMail's `null` — an early version did, and the tone filters silently emptied after every run.

5 · What went wrong, and what we did

Most of these share a shape: **the API accepted the request and reported success, while doing nothing**. That failure mode drove almost every design decision in the project.

Silent failures in the public API

SYMPTOM	CAUSE AND FIX
A campaign went live and sent nothing	A sequence ending on an email step accepts leads and never sends. Every sequence now ends on a wait step.
7 of 17 mailboxes vanished from the app	<code>first: 100</code> is silently capped at 10 on several collections. Our sync read 10, assumed the rest were deleted, and removed them. Fixed with real paging and a skip-delete guard.
Campaign had no sending window	<code>updateCampaignAutomation</code> rejects a display time-zone label and accepts only IANA. The rejection was non-fatal, so creation "succeeded".
Prospects received <code>{{firstName}}</code> literally	Only two merge tags actually resolve, and bodies must be HTML. Tags are now mapped and unknown ones flagged rather than stripped.
Campaign attached to a dead mailbox	A de-authorized mailbox attaches happily and sends nothing. The <code>authorized</code> flag is synced and blocks creation.
Going live refused with no error	The plan caps live campaigns at 20. The refusal arrives as HTTP 200 with an <code>error</code> field, so it read as success.

The wall: no replies, no sending, no trigger

Three capabilities the product needed are absent from every public API. We verified this rather than assuming it — the probe scripts are in the repo so the conclusion can be re-checked, not just trusted.

NEEDED	PUBLIC API	WHAT WE DID
Read reply content	Counts only, no bodies	Read QuickMail's own operation out of their JS bundle; call it through the signed-in browser.
Send a reply	No send mutation exists	<code>replyToEmail</code> via the same session — threaded, from the receiving mailbox.
Set trigger & go live	No trigger field at all	Playwright clicks the real UI, then verifies through the API afterwards.
Stop a prospect's sequence	Not exposed	<code>setProspectsAsDoNotContact</code> + <code>cancelProspects</code> .

Why not scrape the UI for everything? Because clicking is fragile and slow. We use the browser only for the *session*, and issue real GraphQL calls inside it. Clicking is reserved for the two things with no operation at all — the trigger and the live toggle.

Browser automation, and its own problems

- **Introspection is disabled** on the internal endpoint, so input shapes had to be read from the call sites that build them — not guessed, and not learned by pressing Send and watching what fired, which would have

emailed a real prospect.

- **Edge ignores** `--remote-debugging-port` if any instance is already running, and blocks it on the implicit default profile. Every session needs a full restart with an explicit profile path.
- **Angular pre-renders hidden popovers**, so selectors matched invisible duplicates. Every selector is `:visible` -scoped.
- **Answered threads leave the active scope**, so the mirror froze on exactly the threads someone had acted on. The send path now re-reads its own thread immediately.
- **A dropped connection killed a ten-minute sync** at thread 240. Transport failures now retry; GraphQL errors still surface at once, because retrying those could send a second email.

Data quality traps

- QuickMail's `content` field is *not* plain text — it is a full Outlook document, and its own extraction drops content: a reply signed `Best,<br>Amelia` came back as `<br>Amelia`. We parse the HTML body instead.
- Names and subjects arrive entity-encoded — `0'Connor` rendered literally until decoded on the way in.
- Quoted history is stripped before display and before drafting; otherwise the model answers a message from three weeks ago.

6 · The AI layer, honestly

AI is used for three things: classifying inbound replies, drafting answers, and turning a brief into a campaign spec. Each task picks its own provider, so one dead key degrades one feature rather than all of them.

No AI provider is currently reachable. The Gemini quota is exhausted, `ANTHROPIC_API_KEY` is an empty placeholder, and Ollama is not running. Every classification in the inbox today comes from the offline rule-based classifier — which is why making that classifier good mattered more than tuning a prompt.

Offline classifier accuracy

46 real replies from this account, labelled by reading them



`npm run eval-classify` re-runs the check. Nine of the 46 cases came from sampling replies the rules were *not* written against — the first rewrite scored 100% on its own examples while still misreading "no longer with Balboa Retail Partners".

What actually moved the number

PROBLEM	EXAMPLE THAT EXPOSED IT
Curly apostrophes	Outlook autocorrects ' to ’, so every pattern with a straight quote missed. "Let's connect on Friday at 5:00 pm" — with a Meet link attached — read as neutral.
Single keywords	<code>book</code> matched "we already booked a vendor"; <code>remove</code> matched "remove the broken link"; <code>no need</code> matched "no need for pricing details in the first message".
Order of checks	"Not interested, please remove me from your list" is both a rejection and an opt-out. The opt-out is the one with legal weight, so it is settled first.
Deferral treated as rejection	"Try me again next quarter" is a maybe. Suppressing it discards a live prospect.

Every offline verdict says so in its own reasoning field, rather than passing itself off as an AI judgement.

7 · The gate — and the number that matters

Two actions leave the building: sending a reply, and barring an address. Neither happens on one click, and neither happens automatically.

ACTION	WHAT IT DOES	GATE
Send reply	Real email, threaded, from the mailbox that received it.	Panel restating mailbox, recipient, subject and full body. Blocked if suppressed or do-not-contact.
Stop emailing	Bars locally, marks do-not-contact in QuickMail, cancels the in-flight sequence.	Panel spelling out that it spans every campaign and is not undoable from that screen.

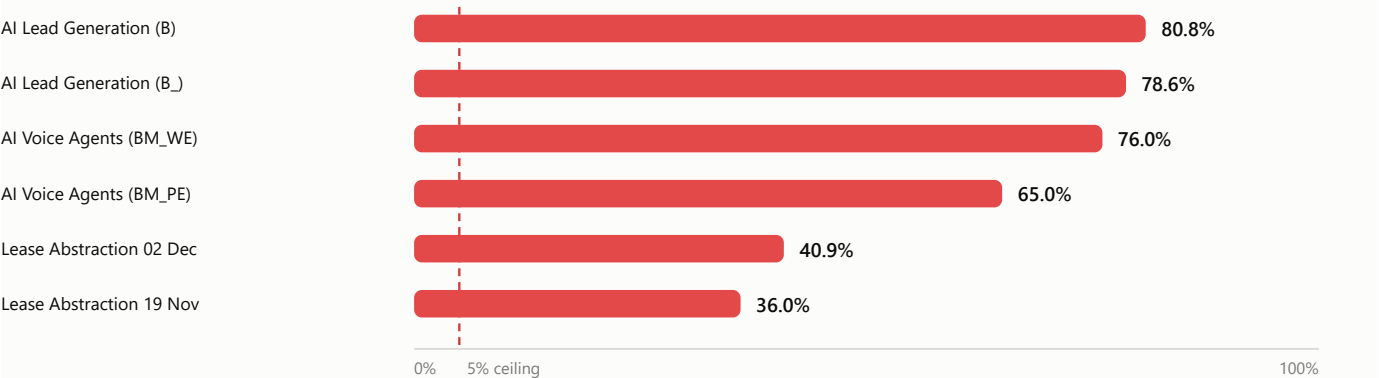
Both write an audit record of who approved what. `npm run unstop` is the escape hatch — it clears QuickMail first and only then local state, the opposite order to stopping, because the safe failure runs the other way.

Deliverability is the real problem

401,428 emails sent	42,339 bounced 10.5% overall	258 replies recorded	9 addresses suppressed far too few
-------------------------------	---	--------------------------------	---

Worst campaigns by bounce rate

Campaigns with more than 100 sends · 5% is the accepted ceiling

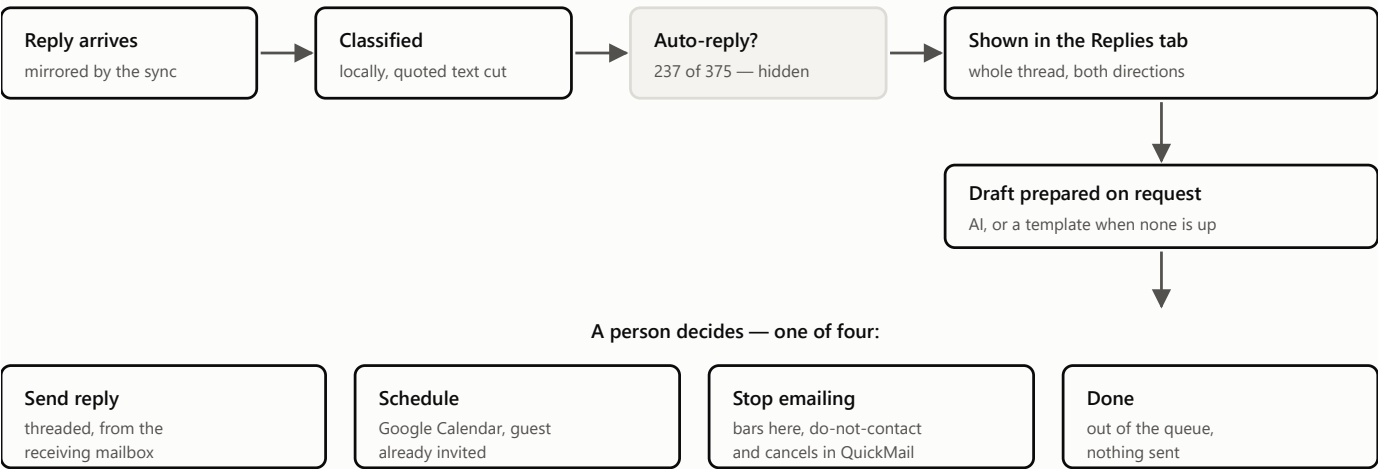


Sending to lists this bad burns the sending domains for every other campaign. The suppression list — 9 entries against 42,339 bounce events — is the gap: a bounce export from QuickMail would seed it properly.

8 · Handling replies — today and next

A reply is the only moment in outbound where a human is definitely needed. Everything before it is volume; everything after it is a conversation. The design point is that the app should do all the preparation and none of the deciding.

How a reply is handled today



The app narrows 375 threads to the 138 a person should look at, and prepares a draft for each. It never chooses between the four boxes.

Where this goes next

IDEA	EFFORT	WHAT IT BUYS
Bulk-action the opt-outs	small	16 explicit opt-outs are queued one at a time. A reviewed multi-select — still one confirmation, but over a list — clears a compliance backlog in a minute instead of an afternoon.
Reply webhook	small	Replies currently appear when someone syncs. A public tunnel to the existing Zapier endpoint makes them arrive in seconds, and the queue reflects reality without anyone pressing a button.
Suggested action, not just a draft	medium	The classification already implies the next move. Pre-selecting it — "this looks like a stop", "this looks like a meeting" — makes the queue a confirm-or-correct list rather than a read-and-decide one.
Learn from corrections	medium	Every time someone overrides a classification, that is a labelled example. Feeding those back into the eval set means the classifier improves from use instead of from us guessing.
Thread-aware drafting	medium	Drafts see only the latest inbound message. Giving the model the whole exchange — and what the campaign originally offered — stops it re-pitching something already discussed.
Follow-up reminders	medium	A positive reply that nobody answers is worse than no reply. A thread that stays open past a few days should resurface.
Send scheduling	small	Approved replies go out immediately. Holding them for business hours in the prospect's zone is a better first impression.

9 · Where the product goes

Ordered by what is holding the account back, not by what is interesting to build. The first group is the difference between a tool that works and one that is trusted.

First — fix what is actively costing us

WORK	WHY IT IS FIRST
Seed the suppression list properly	9 entries against 42,339 bounce events. A CSV export from QuickMail, loaded through the existing importer, stops the same dead addresses being re-imported and re-sent to — roughly 41 bounces per unique bad address today.
Pre-send list hygiene	A campaign at 80.8% bounce should never have sent. Verifying addresses before enrolment, and refusing to launch above a threshold, protects the sending domains every other campaign depends on.
Clear the 38 stop requests	People who asked to be left alone are still enrolled. This is the one item with legal weight.
A working AI provider	Everything AI-shaped is currently running on rules. A funded key restores drafting and classification together.

Then — make the daily work faster

- **Deliverability dashboard.** Bounce rate per campaign and per sending domain, with a warning before a campaign goes live rather than a post-mortem after it burns a domain.
- **Campaign templates from what worked.** 359 campaigns and 258 replies is enough signal to say which openers earn answers. Right now that lives in nobody's head.
- **Scheduled sync.** The reply mirror refreshes when someone remembers. A timed job, or the webhook above, removes that dependency.
- **Mailbox health.** 5 of 17 mailboxes fail sender assignment and it is discovered at campaign-creation time. Checking continuously would surface it before it blocks anyone.

Later — worth doing, not yet urgent

- **Reduce the QuickMail-internal dependency.** Reading replies straight from the sending mailboxes over IMAP would remove the undocumented API from the critical path — at the cost of credentials for 17 mailboxes across 13 domains. Worth revisiting if QuickMail ever breaks the internal endpoint.
- **Multi-user.** One shared login today. Real accounts would make the audit trail — already written on every send and stop — actually attributable.
- **Move off SQLite.** Fine for one machine and one operator. A second concurrent user is the trigger to reconsider, not a date.
- **Automated tests.** There is no test framework; correctness is currently enforced by the eval harness for classification and by verifying against QuickMail after every write. That has caught real bugs, but it does not scale.

What we are deliberately not doing. Sending without a human click, and suppressing without one. Both have been asked for as time-savers, and both remove the only check that has caught our own mistakes — including a draft that

looked fine and was written by a template because no model was running.

10 · Where it stands

Working and verified

- Campaign creation through the v2 API, including the sequence, senders, sending window and enrolment.
- Trigger and go-live via browser automation, verified through the API afterwards.
- Full reply mirror — 375 threads, 1,127 messages.
- Sending a real threaded reply. **Confirmed end to end**: a test reply reached its recipient and appears in the live thread.
- Stop emailing: local suppression plus QuickMail do-not-contact and sequence cancellation, confirmed by reading the prospect back.
- Search, tone filters, and a sync button in the Replies tab.
- Offline classification at 100% on the labelled set.

Open

- **38 threads asking to stop**, 16 of them explicit opt-outs, all waiting on a human click.
- **No AI provider reachable** — needs a working key or a local model.
- Suppression list holds 9 entries against 42,339 bounce events; needs the CSV export.
- 5 of 17 mailboxes hit an HTTP 500 on sender assignment — a support ticket is drafted.
- The reply webhook needs a public tunnel to be reachable from Zapier.
- 7 orphan campaigns from failed creation runs need manual deletion.

Dependencies worth stating out loud

DEPENDENCY	CONSEQUENCE IF IT GOES
Edge running with <code>--remote-debugging-port=9222</code> , signed in	Replies, sending and stopping all stop working. They report the reason rather than failing quietly.
QuickMail's internal API staying as it is	Same three features break. Campaign creation is unaffected — that uses the public API.
A reachable AI provider	Drafting falls back to a fixed template and classification to rules. Both say so on screen.
The 20 live-campaign plan cap	New campaigns cannot go live until old ones are paused.

Repository: github.com/abhishek-sybrant/mail-agent · 18,984 lines across app, scripts and schema · 17 database migrations.